INDEX

ANITS

S.No	Date	Name of the Experiment	Mapping To Course Outcome	Grade	Signature
1)	8/7/26	Introduction to HTML and CSS	CO1	10	
2)	15/7/26	Static webpage using HTML and CSS	CO1	9	
3)	22/7/26	HTML codes using audio & video tags	CO1	10	
4)	29/7/26	Program using Javascript	CO1	10	
5)	5/8/26	Implement modules in NodeJS	CO2	10	
6)	12/8/26	Develop mini application using Express & NodeJS	CO2	10	
7)	19/08	Code Tantra (static webpage)	CO_1	10	
8)	02/09	Code Tantra (portfolio)	CO_1	10	
9)	09/09	Code Tantra (Simple Calculator)	CO_1	10	

Scanned with OKEN Scanner

INDEX

ANITS

S.No.	Date	Name of the Experiment	Mapping To Course Outcome	Grade	Signature
10	9/9	Create a function & class	CO2	10	
11	23/9	Implement Http methods	CO2	10	
12	30/9	Develop CRUD operation	CO3	10	
				10	

Anil Neerukonda Institute of Technology & Sciences
(Autonomous)
Affiliated to AU, Approved by AICTE & Accredited by NAAC with A+ Grade
Accredited by NBA (B.Tech – ECE, EEE, CSE, IT, MECH, Civil & Chemical)
Sangivalasa-531 162, Bheemunipatnam Mandal, Visakhapatnam District
Phone: 8712005999, 8712000222. Website: www.anits.edu.in, email:

Department of Computer Science & Engineering (AI & ML)

Academic Year:	2026-2027	**Semester:**	3-1
Name of the Student:	J. Vishnupriya	**Roll Number:**	A24126552143
Name of the Lab:	FS lab	**Timings:**	9:40-12:10

LAB WEEKLY ASSESSMENT SHEET

Date	Experiment /Week No	Observation Book				Total (15 Marks)	Record (10 Marks)	Faculty Remarks
		Theoretical Knowledge (4 Marks)	Programming Knowledge (4Marks)	Results (5Marks)	Viva-Voce (2Marks)			
1/7	01	4	4	5	2	15		
8/7	02	4	3	5	2	14		
15/7	03	4	4	5	2	15		
22/7	04	3	4	5	2	14		
29/7	05	4	4	4	2	14		
5/8	06	4	4	5	2	15		
12/8	07	4	4	5	2	15		
19/8	08	4	4	5	2	15		
2/9	09	4	3	4	2	14		
9/9	10	4	4	4	2	15		
23/9	11	4	4	5	2	15		
30/9	12	4	4	5	2	15		

Name of the Faculty: Ch. Sarathi

Faculty Signature: Sarathi

1/7/26

Introduction to HTML and CSS

→ HTML (Hyper text Markup language) is the standard markup language used to create and structure content on web pages. It defines the basic structure of a webpage using elements and tags such as headings, paragraphs, links, images, lists, tables, forms and other multimedia content. HTML documents are interpreted by web browsers to display content in a structured and meaningful manner. HTML provides the structure and semantic organization of a webpage.

→ CSS (cascading style sheets) is a stylesheet language used to control the presentation and visual appearance of HTML documents. It is used to define properties such as colors, fonts, backgrounds, spacing, borders, alignment, dimensions and layouts. CSS also supports responsive design, allowing webpages to adapt to different screen sizes and devices. CSS can be applied using inline, internal or external stylesheets.

→ HTML and CSS work together to develop webpages, where HTML provides the structure and content, while CSS provides the styling, layout, visual presentation. The combination of both forms

Scanned with OKEN Scanner

the foundation of modern web development and enables the creation of organized, attractive and user-friendly web interfaces.

Difference between MERN stack and FULL Stack

→ MERN stack is a Javascript-based technology stack used for developing modern web applications. MERN stands for MongoDB, Express.js, React.js, Node.js. Mongo DB is used for database management, Express.js handles the backend framework and API's, React.js is used to develop frontend user interface, and Node.js provides the server-side runtime environment.

→ Full-stack Development refers to the development of both the frontend and backend of a web application along with database integration. A full stack developer works with technologies used for designing the user interface, implementing server-side logic managing databases.

8/7/26

HTML Components:

1. HTML tag - Define the structure and elements of a webpage
2. Headings - Used to display titles and headings using <h1> to <h6>
3. Paragraphs - Used to display text using the <p> tag
4. Links - Used to navigate between webpages using <a> tag
5. Images - Used to display images using <image> tag
6. Lists - Used to create ordered and unordered lists using <ol>, <ul>,
7. Tables - Used to organize data in columns & rows.
8. Forms - Used to collect user input using elements such as text boxes, buttons and checkboxes.
9. Div & Span - Used to group and organize webpage content
10. Semantic Elements - Elements such as <header>, <nav>, <section>, <article>, <footer> webpage structure.

Scanned with OKEN Scanner

CSS Components:

1. Selectors - Select HTML elements to apply styles
2. Properties - Define the visual characteristic of elements.
3. Values - Specify the setting of a CSS property.
4. Colors - Define text, background, border colors
5. Fonts - Control font family, size, style and weight.
6. Backgrounds - Set background colors and images
7. Borders - Define the border style, width and radius.
8. Margin - Controls the space outside an element.
9. Padding - Controls the space inside element
10. Layout - Controls the arrangement of elements using Flexbox, grid and positioning

HTML vs HTML 5

HTML	HTML 5
older version of HTML	latest major version of HTML.
Limited multimedia support	supports audio and vide
uses external plugins for multimedia	No external plugs needed.
Has fewer semantic tags	Provides semantic tags
No <canvas> element	supports <canvas>
Limited web storage	Supports local storage
Complex DOCTYPE declaration	Simple <!DOCTYPE HTML

CSS vs CSS3

CSS	CSS 3
Earlier version of CSS	Advanced version of C
Basic styling features	Provides advanced styli features.
Limited animations	Supports animations and transitions.
No flexible layout systems.	Supports Flexbox and gri
Limited visual effects	Supports shadows, gradients effects.
No media queries	Supports media queries
Uses a single specification	Divided into different modules.

Scanned with OKEN Scanner

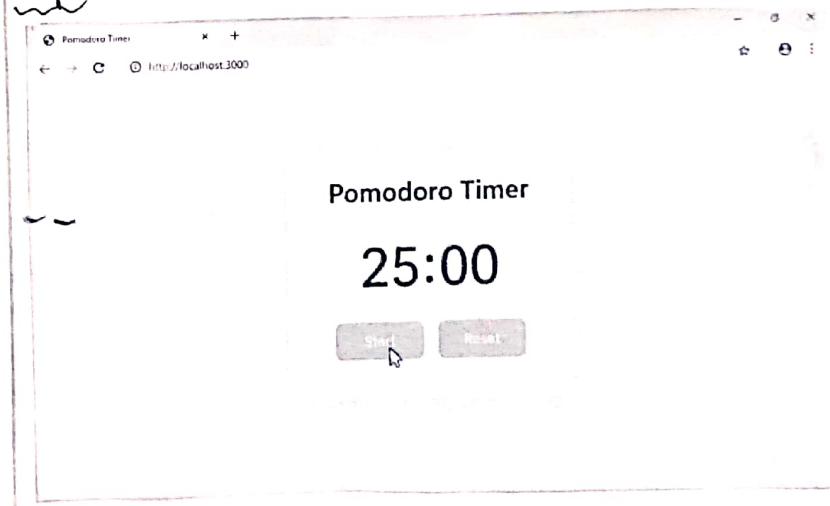
15/9/26

Design a static webpage using HTML and CSS

Pomodoro Timer, using HTML, CSS.

```
<!DOCTYPE html>
<html>
<head>
  <title> Pomodoro Timer </title>
  <style>
    body{
      font-family : Arial;
      background : # f5f5f5;
      text-align : center;
      padding : 50px;
    }
    .timer {
      width : 300px ;
      margin : auto;
      padding : 30px;
      background : white ;
      border-radius : 15px;
      box-shadow : 0 0 10px gray ;
    }
    h1 {
      color : #e74c3c;
    }
    .time{
      font-size : 50px;
      margin : 20px ;
    }
    button {
      padding : 10px 20px ;
      margin : 5 px ;
      border : none;
      border-radius : 5 px ;
      background : # e74c3c;
      color : white ;
    }
  </style>
</head>
<body>
  <div class = "Timer">
    <h1 > Pomodoro Timer </h1>
    <div class = "time">25:00 </div>
    <button > Start </button>
    <button > Reset </button>
  </div>
</body>
</html>
```

output :-



Scanned with OKEN Scanner

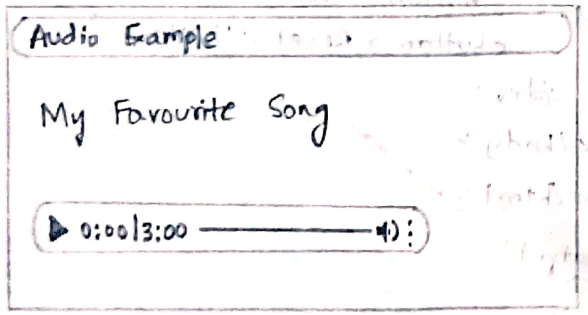
22/07/2026

i) Design an HTML code using audio tags.

Code:

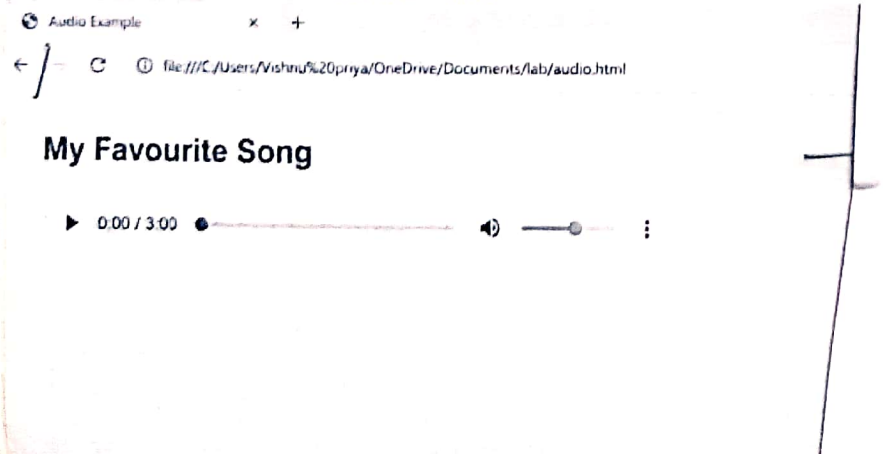
```
<!DOCTYPE html>
<html>
<head>
   <title> Audio Example </title>
</head>
<body>
   <h2> My Favourite Song </h2>
   <audio controls>
      <source src = "song.mp3" type = "audio/mpeg">
      Your browser does not support the audio tag
   </audio>
</body>
</html>
```

output:



Steps:

- New HTML file

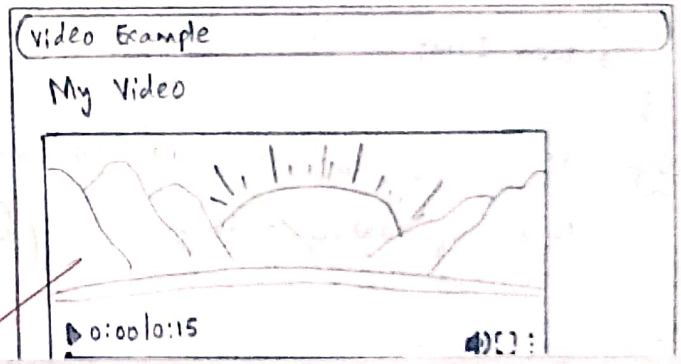


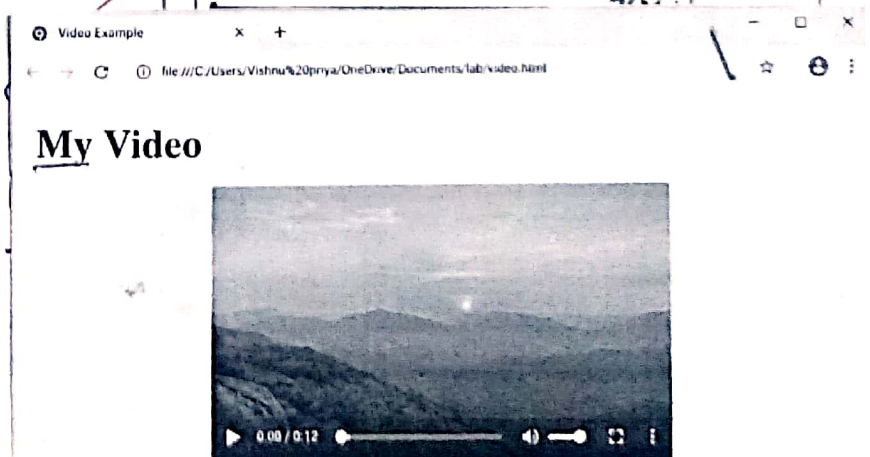
ii) Design a HTML ode using video tags.

Code:

```
<!DOCTYPE html>
<html>
<head>
   <title> video Example </title>
</head>
<body>
   <h2> My video </h2>
   <video width = "400" controls>
      <source src = "video.mp4" type = "video/mp4">
   </video>
</body>
</html>
```

output:





Scanned with OKEN Scanner

Write a program using Javascript (arrays, functions)

Code:

```
let numbers = [12,45,8,23,56,19,34,67,90,11];
function displayArray(arr){
    console.log("Array Elements :");
    for(let i=0 ; i< arr.length ;i++){
        console.log(arr[i]);
    }
}
function findSum(arr){
    let sum = 0;
    for(let i=0; i<arr.length ;i++){
        sum += arr[i];
    }
    return sum;
}
```

Op: 12 45 8 23 56 19 34 67 90 11

Sum of array = 365

Experiment - 4

4. Develop dynamic webpage using Javascript

Program:-

```
<!DOCTYPE html>
<html>
<head>
<title> Dynamic webpage </title>
<style>
 body{
text-align : center;
font-family: Arial;
margin-top: 100px;
 }
 button{
padding : 10px 20px ;
font-size : 16 px ;
cursor : pointer;
 }
</style>
</head>
<body>
<h1 id = "heading" > welcome to my webpage </h1>
<p id= "message" > click the button to change this
 webpage </p>
<button onclick = "change page()" > clickMe </button>

<script>
function changepage(){
 document.getElementById("heading").innerHtml
 = "welcome to javascript !" ;
```

Scanned with OKEN Scanner

```
document.getElementById("heading").innerHTML
= "The webpage has been changed
dynamically.
document.body.style.backgroundColor = "lightblue
}
</script>
</body>
</html>
```

Output:

Dynamic Webpage

Welcome to My webpage

Click the button to change this webpage

Click Me

→ When the user clicks "Click Me" the javascript function changepage() is called, then the heading dynamically changes and background also change.

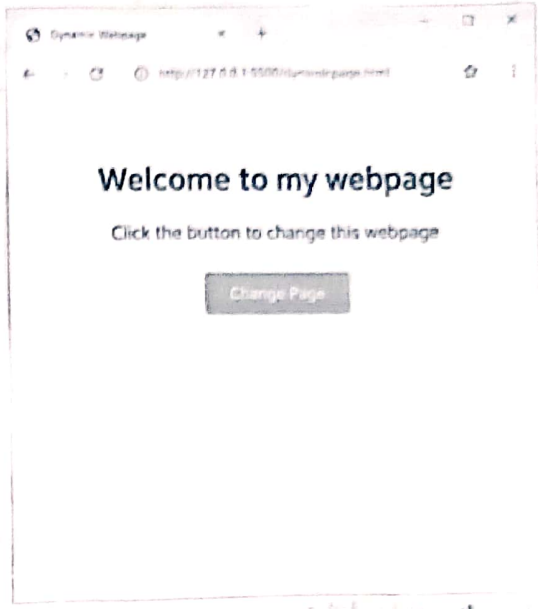
→ After clicking the button:

Dynamic Webpage

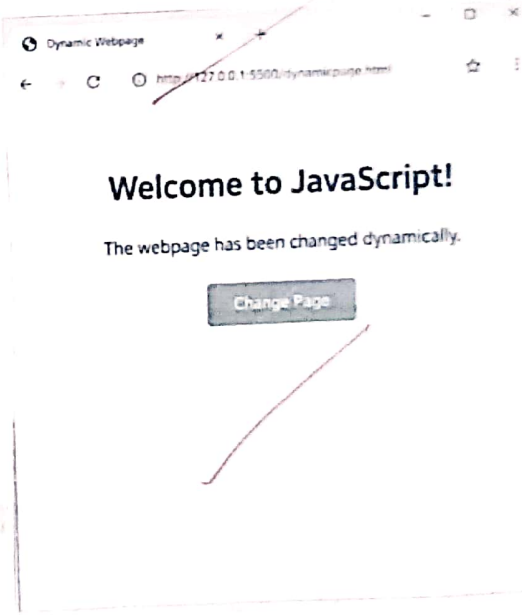
Welcome to My webpage

The webpage has been changed dynamically

Click Me



Before clicking the button



After clicking the button

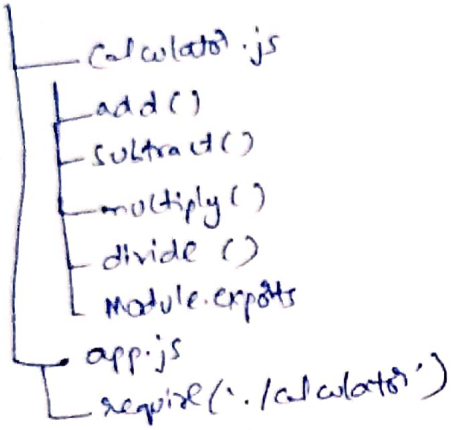
Scanned with OKEN Scanner

Experiment 5

(5) Implement Modules in Node.JS.

Aim:- To develop a Node.js application using modules to perform arithmetic operations

Node.js Modules



(1) calculator.js - Module

```
function add(a,b){
 return a+b;
}
function subtract(a,b){
return a-b;
}
function multiply(a,b){
return a*b;
}
function divide(a,b){
return a/b;
}
module.exports = { add, subtract, multiply,
                   divide };
```

(2) app.js - Main Program

```
const calculator = require('./calculator');
console.log("Addition:", calculator.add(20,10));
console.log("Subtraction:", calculator.subtract
(20,10));
console.log("Multiplication:", calculator.
multiply(20,10));
console.log("Division:", calculator.
divide(20,10));
```

(3) index.html

```
<!DOCTYPE html>
<html>
<head>
<title> Node.js Modules </title>
<style>
body{
font-family : Arial;
text-align: center;
margin-top: 50px;
}
.box{
width : 400px;
margin: auto;
padding : 30px;
border : 1px solid #ccc;
border-radius : 10px;
}
input, select, button{
padding : 10px;
margin : 8px;
}
```

Scanned with OKEN Scanner

```
button{
cursor:pointer;
}
#result{
margin-top:20px;
font-size:20px;
}
</style>
</head>
<body>
<div class="box">
<h1>Node.js Modules</h1>
<input type="number" id="num1" placeholder
="First Number">
<input type="number" id="num2" placeholde
="Second Number">
<br>
<select id="operation">
<option value="add">Addition</option>
<option value="Subtract">Subtraction</option>
<option value="multiply">Multiplication</optio
<option value="divide">Division</option>
</select>
<br>
<button onclick="calculate()">Calculate</butto
<div id="result"></div>
</div>
```

```
<script>
function calculate(){
let a = Number(document.getElementById("num1"
.value);
let b = Number(document.getElementById("num2"
.value);
let operation = document.getElementById("operation
.value;
let result;
if (operation === "add")
result = a+b;
else if (operation === "Subtract")
result = a-b;
else if (operation === "multiply")
result = a*b;
else
result = a/b;
document.getElementById("result").innerHTML
= "Result :" + result;
}
</script>
</body>
</html>
```

output:-

Node app.js

Addition : 30

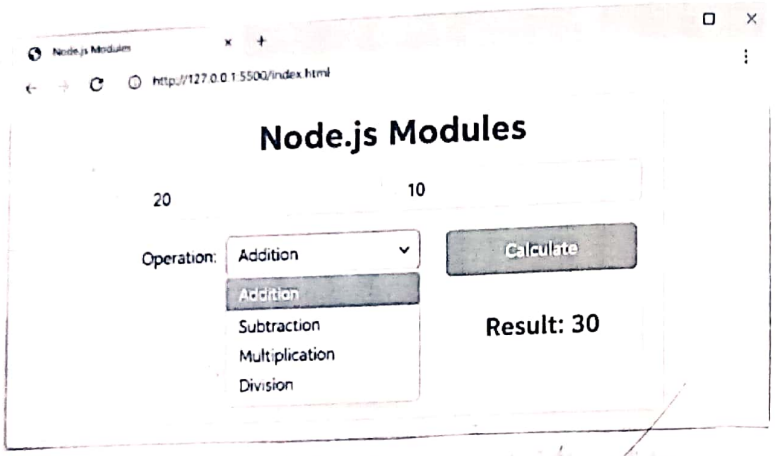
Subtraction: 10

Multiplication : 200

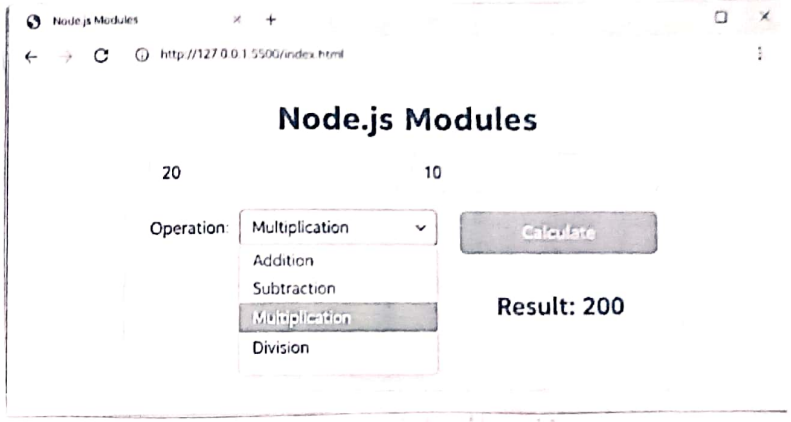
Division : 2

Scanned with OKEN Scanner

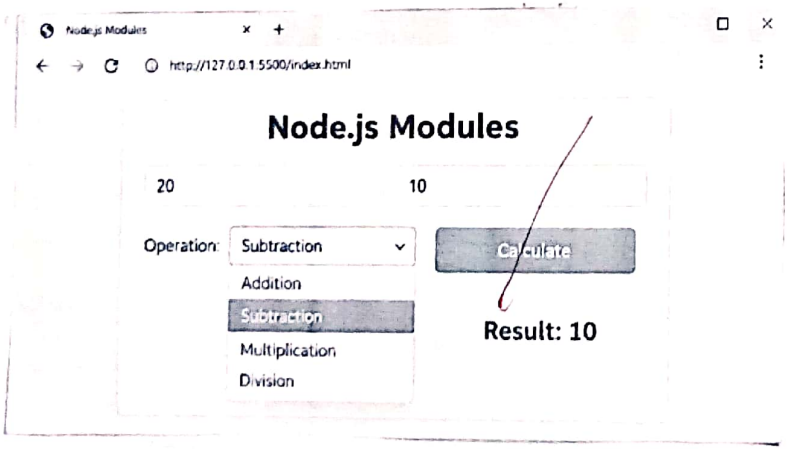
Node.js Modules - Exploring All operations



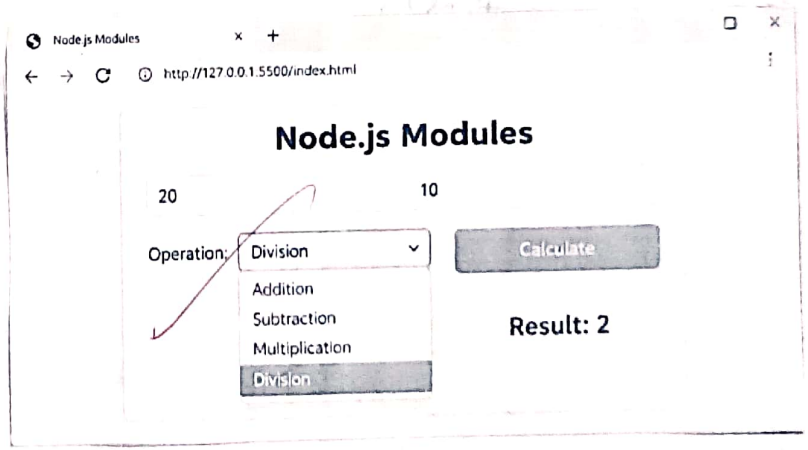
1. Addition



3. Multiplication



2. Subtraction



4. Division

Scanned with OKEN Scanner

Experiment-6

⑥ Develop Mini application using Express and Node.js

Aim: To develop a mini login application using Node.js and Express.js that validates the username and password entered by the user.

server.js :-

```
const express = require("express");
const cors = require("cors");
const app = express();
const corsOptions = {
origin: "http://localhost:3000",
methods: ["GET", "POST", "PUT", "DELETE"],
allowedHeaders: ["Content-Type", "Authorization"]
};
app.use(cors(corsOptions));
app.use(express.json());
app.use(express.urlencoded({extended: true}));
app.use(express.static("public"));
app.post("/login", (req, res) => {
const { username, password } = req.body;
if (username === "admin" && password === "1234")
{
  res.json({
  success: true,
  message: "Login successful"
  });
}else {
res.json({
success: false,
message: "Invalid username or password"});
}
});
app.listen(3000, () => {
console.log("server running at http://localhost
:3000");
});
```

② public/index.html

```
<!DOCTYPE html>
<html>
<head>
<title>Login Page</title>
<style>
body{
font-family: Arial;
background-color: lightblue;
}
.login{
width: 300px;
margin: 100px auto;
padding: 25px;
background-color: white;
text-align: center;
border-radius: 10px;
}
input{
width: 90%;
```

Scanned with OKEN Scanner

```
    padding: 10px;
    margin: 10px;
}
button {
    padding: 10px 20px;
    background-color: blue;
    color: white;
    border: none;
    cursor: pointer;
}
#message {
    margin-top: 15px;
}
</style>
</head>
<body>
<div class="login">
<h2>Login Here</h2>
<input type="text" id="username" placeholder
="Enter Username">
<input type="password" id="password" placeholder
="Enter password">
<button onclick="login()">Login</button>
<p id="message"></p>
</div>
<script>
function login(){
const username = document.getElementById("username
.value;
const password = document.getElementById("password
.value;
fetch("http://localhost:3000/login", {
method: "POST",
headers: {
"Content-Type": "application/json"
},
body: JSON.stringify({
username: username,
password: password
})
})
.then(response => response.json())
.then(data => {
document.getElementById("message").textContent
= data.message;
})
.catch(error => console.log(error));
});
}
</script>
</body>
</html>
```

folder structure :-

```
login/
├── server.js
├── package.json
├── package-lock.json
├── node_modules/
└── public/index.html
```

Scanned with OKEN Scanner

Execution:-

Install the required packages:

```
npm init -y
npm install express cors
```

Run the server:

```
node server.js
```

output in the terminal:

```
Server running at http://localhost:3000
```

Open the browser:

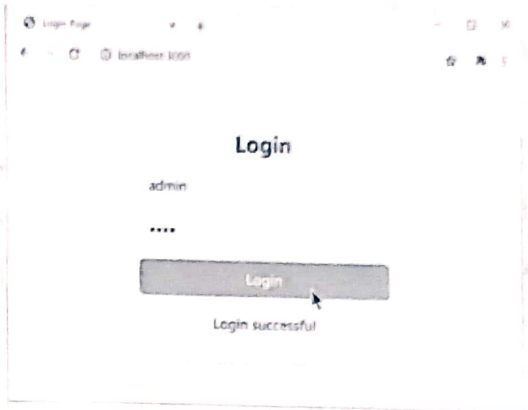
```
http://localhost:3000
```

Output:-

→ The login application was executed successfully using Node.js and Express.js

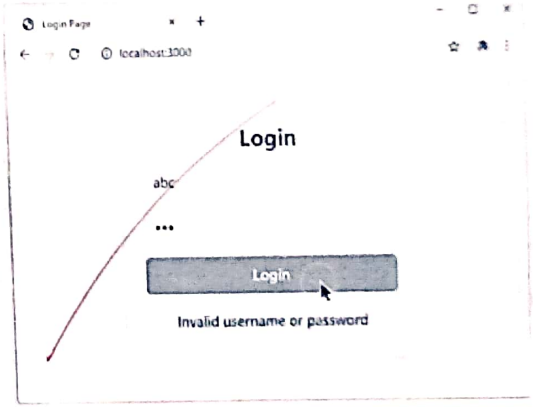
→ The user enters a username and password and the login details are sent to Express server using a POST request.

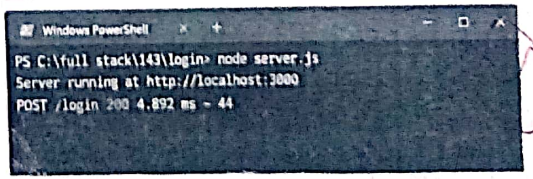
→ The server validates the credentials and returns the appropriate message





Case1: Successful login
(username: admin | password: 1234)





Case 2: Invalid login
(username: abc | password: 123)

Scanned with OKEN Scanner

Experiment-7

⑦ Create a functional component and class component using react.

Aim:- To develop a react application using Functional components and class components to display student details, manage state, handle button clicks and dynamically update data on a webpage.

Functional Component:-

A functional component is a javascript function that returns JSX to display the user interface. It uses React Hooks such as usestate to manage state and update the webpage dynamically.

Program:-

```
import React, {use State} from
'react':
function StudentFunctional (){
    const [name, setName] = useState("Vishnu");
    const [course, setCourse] = useState("CSM");
    const [count, setCount] = Usestate(0);
    const changeName = () => {
        setName("Vishnupriya");
    };
    const changeCourse = ()=>{
        setCourse("CSD");
    };
    const incrementCount = ()=>{
        setCount(count+1);
    };
    const incrementCount = ()=>{
        setCount(count+1);
    };
    return (
    <div style={{
        border: "2px solid blue",
        padding: "20px",
        margin: "20px",
        borderRadius: "10px",
        textAlign: "center"
    }}>
    <h1>Student Details</h1>
    <h2>Functional Component</h2>
    <p><b>Name :</b> {name}</p>
    <p><b>Course :</b> {course}</p>
    <p><b>Button clicks:</b>{count}</p>
    <button onClick={changeName}>
        changeName</button>
    <button onClick={changeCourse}>change
        course</button>
    <button onClick={incrementCount}>
        Increment</button>
    </div>
```

Scanned with OKEN Scanner

Output :-

When the user clicks the buttons, the name course and count change dynamically without reloading the webpage.



- Class Component :-

A class component is an ES6 JavaScript class that extends React Component. It uses the render() method to return JSX and manages state using this.state and this.setState().

Program :-

```
import React, {Component} from
'react';
class studentclass extends Component{
    constructor(props) {
        super(props);
        this.state ={
            name: "vishnupriya",
            course: "computer science",
            count = 0
        };
    }
    changeName = () => {
        this.setstate ({
            name:" Vishnu"
            });
        };
    changeCourse = () => {
        this.setstate ({
            Course : " CSM "
            });
        };
```

Scanned with OKEN Scanner

```
incrementCount = () => {
  this.setState((prevState) => {
    count: prevState.count + 1
  });
};
render() {
  return (
    <div style={{
      border: "2px solid green",
      padding: "20px",
      margin: "20px",
      borderRadius: "10px",
      textAlign: "center",
    }}>
      <h1>Student Details</h1>
      <h2>Class Component</h2>
      <p><b>Name:</b> {this.state.name}</p>
      <p><b>Course:</b> {this.state.course}</p>
      <p><b>Button Clicks:</b> {this.state.count}</p>
      <button onClick={this.changeName}>
        Change Name
      </button>
      <button onClick={this.changeCourse}>
        Change Course
      </button>
```

localhost:3000

React Components

Student Details

Functional Component

Name: Tejaswari

Course: CSE

Button Clicks: 0

Change Name | Change Course | Increment

Student Details

Class Component

Name: Tejaswari

Course: CSE

Button Clicks: 0

Change Name | Change Course | Increment

Scanned with OKEN Scanner

Experiment-8

⑧ Implement http methods using JSON Server

Aim:- To implement HTTP methods such as GET, POST, PUT, PATCH and DELETE using JSON server to perform CRUD(create, read, update & Delete) operations on student records.

Theory:- JSON server is a tool used to create a fake Rest API using a JSON file. It allows us to perform HTTP operations without developing a backend server from scratch.

→ HTTP methods are used to communicate between a client and a server.

- GET - Retrieves data from the server
- POST - Adds new data to the server
- PUT - Replaces an existing record
- PATCH - Updates selected fields of a record
- DELETE - Removes a record from the server

a. Create a JSON server:-

Step1: Initialize the project

Open the vs code terminal and execute the following commands

```
mkdir JSONServer
cd JSONServer
npm init -y
npm install json server
```

Step2: Create db.json

Create a file named db.json and add the following student data

```
{
  "students": [
    {
      "id": "1",
      "name": "Teju",
      "course": "CSM",
      "marks": 90
    },
    {
      "id": "2",
      "name": "Vishnu",
      "course": "CSM",
      "marks": 90
    },
    {
      "id": "3",
      "name": "Sumi",
      "course": "IT",
      "marks": 88
    }
```

Scanned with OKEN Scanner

Step 1: Start the JSON server

Run the command: json-server --watch db.json

The server start at
http://localhost:3000
The student records can be accessed using:
http://localhost:3000/students

2. Implementation of HTTP methods:

a. GET method: The GET method is used to retrieve student records from JSON server.

Code:

```
fetch('http://localhost:3000/students')
.then(response => response.json())
.then(data => {
    console.log(data);
})
.catch(error => {
    console.log(error);
});
```

Output:

b. POST method: The POST method is used to insert a new student record into the server.

Code:

```
fetch('http://localhost:3000/students', {
  method: 'POST',
  headers: {
    'Content-Type': 'application/json'
  },
  body: JSON.stringify({
    name: 'Rahul',
    course: 'CSE',
    marks: 92
  })
})
.then(response => response.json())
.then(data => {
    console.log(data);
})
```

Scanned with OKEN Scanner

```
.catch (error => {
    console.log(error);
});
```

output:-

C. PUT method: The PUT method is used to replace an existing student record with updated data.

Code:-

```
fetch('http://localhost:3000/students/1', {
    method: 'PUT',
    headers: {
        'Content-Type': 'application/json'
    },
    body: JSON.stringify({
        name: 'Tejaswini',
        course: 'AIML',
        marks: 95
    })
})
.then(response => response.json())
.then(data => {
    console.log(data);
})
.catch(error => {
    console.log(error);
});
```

output:-

D. PATCH method: The PATCH method is used to update only selected fields of an existing student record.

Code:-

```
fetch('http://localhost:3000/students/2', {
    method: 'PATCH',
    headers: {
        'Content-Type': 'application/json'
    },
    body: JSON.stringify({
        marks: 96
    })
})
```

Scanned with OKEN Scanner

```
.then(response => response.json())
.then(data =>{
    console.log(data);
})
.catch(error =>{
    console.log(error);
});
```

output:-

```
.catch(error =>{
    console.log(error)
});
```

output:-

E. DELETE method: The DELETE method is used to remove an existing student record from the server.

Code:-
```
fetch('http://localhost:3000/students/3',{
    method:'DELETE'
})
.then(response=>{
    if(response.ok){
        console.log("student deleted
                    Successfully");
    }
})
```

- After executing the POST, PUT, PATCH and DELETE operations in sequence, the remaining student will be.

output:-

Scanned with OKEN Scanner

Experiment-9

④ Develop CRUD operations using MongoDB.

Aim:- To develop a webapplication using Node, Express.js and MongoDB to perform CRUD operations (create, read, update, delete) on student records.

Theory:- MongoDB is a NoSQL database that stores data in flexible, JSON-like documents. CRUD operations are used to manage data in a database.

Mongoose is an object Data Modeling (ODM) library for MongoDB and Node.js. It helps define schemas, interact with collections and perform database operations.

1. Installation and project setup:-

```
mkdir MongoDB.CRUD
cd MongoDB.CRUD
npm init -y
npm install express mongoose
```

program: server.js

```
const express = require('express');
const mongoose = require('mongoose');
const app = express();
const PORT = 3000;
app.use(express.json());
mongoose.connect('mongodb://127.0.0.1
                  :27017/college')
.then(() => console.log("MongoDB Connected"))
.catch(err => console.log(err));
// create student
const studentSchema = new
mongoose.Schema({
    name: {
      type: String,
      required: true
    },
    course: {
      type: String,
      required: true
    },
    marks: {
      type: Number,
      required: true
    }
});
// create student model
const student =
    mongoose.model('student', studentSchema)
// CREATE Add
app.post('/students', async (req, res)=>{
    try{
    const student = new Student(req.body);
    const savedStudent = await Student.save();
```

Scanned with OKEN Scanner

```
    res.status(201).json(savestudent);
  }catch(error) {
      res.status(400).json({message:
                    error.message});

    }
  });
// READ : Get all students
app.get('/students', async (req, res) => {
  try{
     const  students = await
  Student.find();
     res.status(500).json({message: error.
                              message});

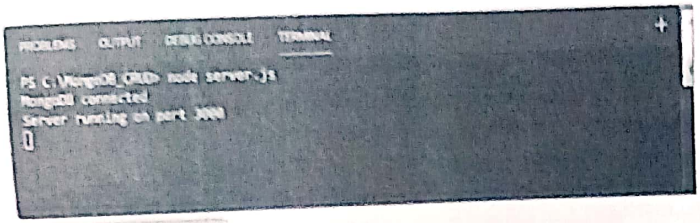
  }
  });
// READ : Get student by ID
app.get('/students/:id', async (req,res)=>{
  try{
     const student = await
        Student.findById(req.params.id);
     if(!student){
        return res.status(404).json({
           message: "student not
                          found"
        });
     }
     res.status(200).json(student);
  } catch(error){
     res.status(400).json({ message: error.
                          message});

  }
  });
//update
app.put('/students/id', async(req, res)=>{
  try{
  const student : await
  Student.findByIdAndUpdate(
     req.params.id,
     req.body,
     {new: true, runValidators: true}

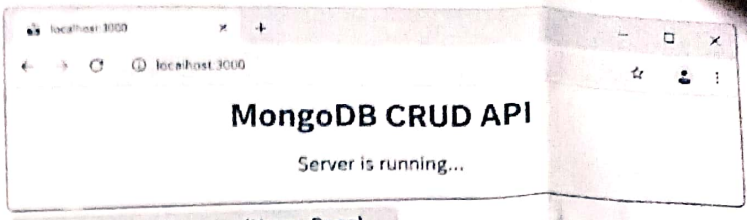
  );
if(!student){
   return res.status(404).json({
      message: "student not found"
   });

  }
  res.status(200).json(student);
} catch(error){
res.status(400).json({ message: error.message});
}
});
//DELETE
app.delete('/students/:id', async (req, res) =>{
   try{
     const student = await student,
```

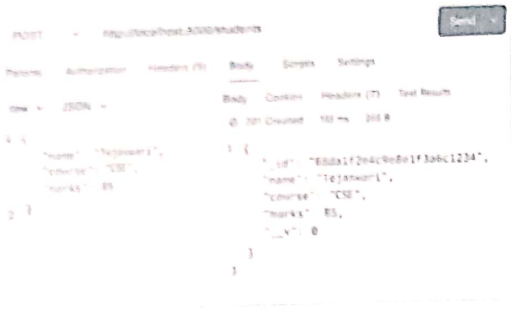
Scanned with OKEN Scanner



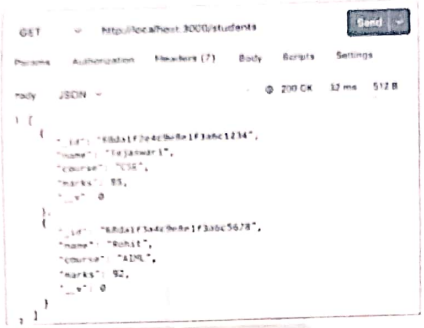
1. Terminal Output



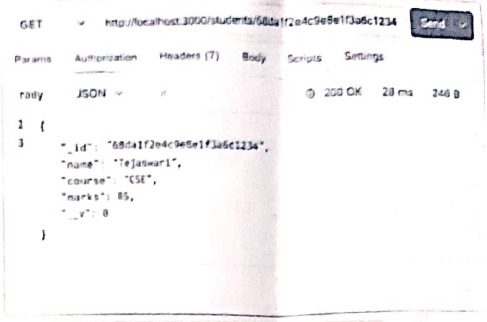
2. Chrome Browser (Home Page)



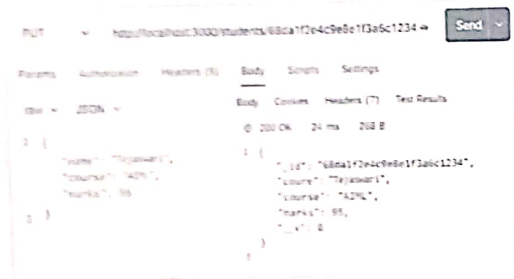
3. Create (POST) – Add Student



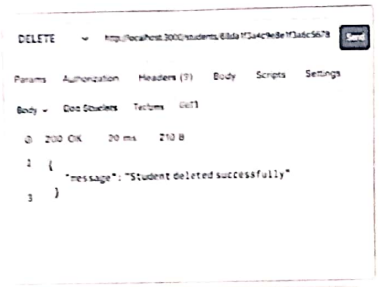
4. Read (GET) – Get All Students



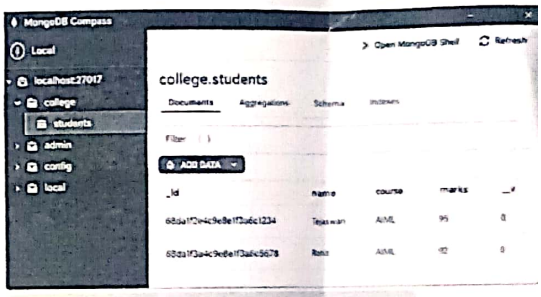
5. Read (GET) – Get Student by ID



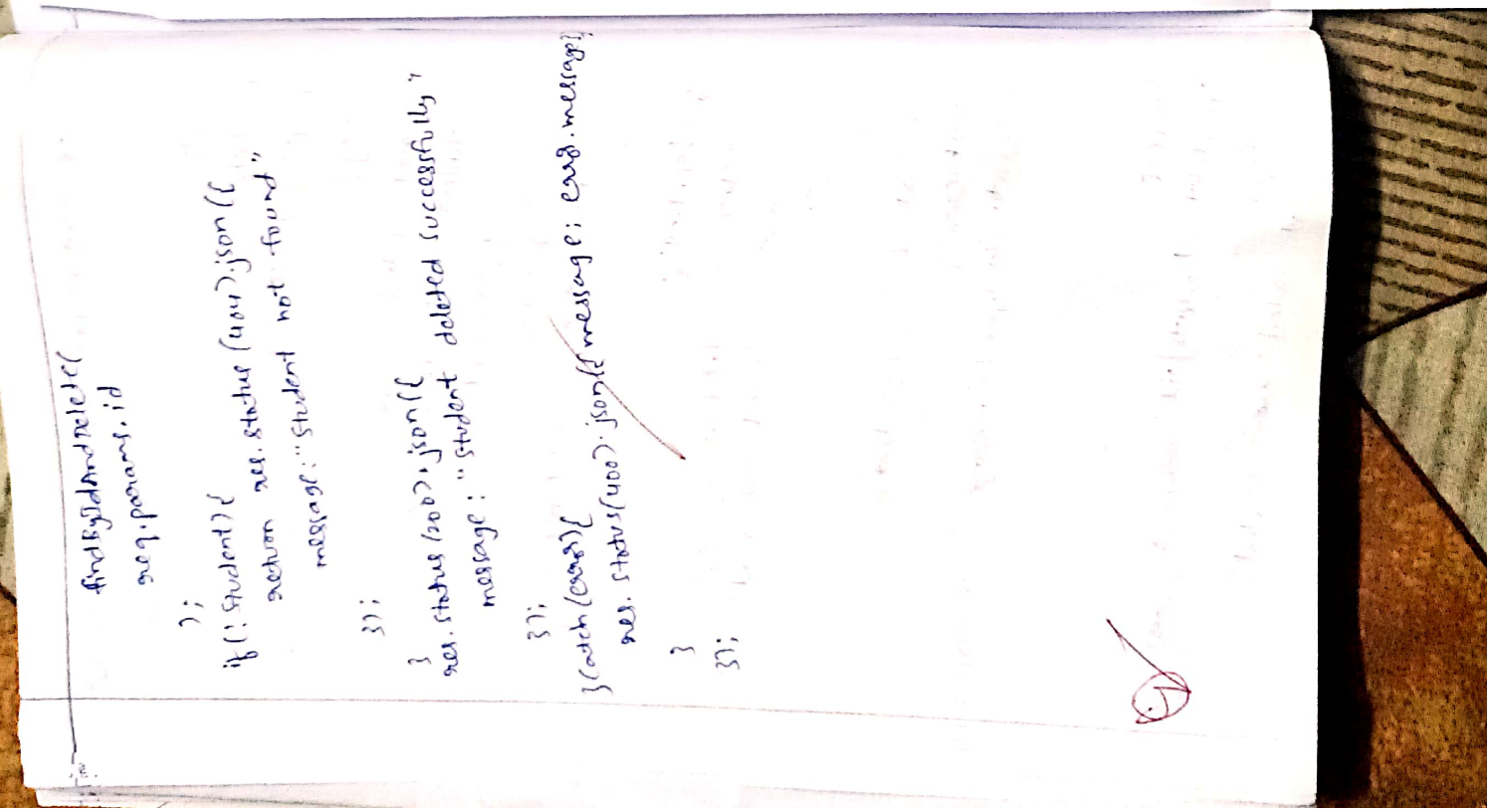
6. Update (PUT) – Update Student



7. Delete (DELETE) – Remove Student



8. MongoDB Compass – Database View



Scanned with OKEN Scanner